

NIVEL FUNDAMENTAL

MÓDULO 1. ENTORNO Y HERRAMIENTAS

AMBIENTES VIRTUALES

Espacios aislados para instalar las dependencias de un proyecto, y evita que no interfieran las dependencias de diferentes proyectos.

Java suele gestionar sus dependencias con Maven o Gradle, mientras que en Python un ambiente virtual aísla las instalaciones de cada proyecto.

Para crear un ambiente virtual con Python, desde la terminal del IDE usar el comando:

```
python -m venv .venv
```

La carpeta “**.venv**” contendrá el entorno aislado.

Poetry

Herramienta para crear proyectos, gestionar ambientes virtuales, administrar dependencias, definir metadatos del proyecto y generar archivos de bloqueo de versiones. Permite que la configuración del proyecto y sus dependencias se encuentren centralizadas. Su configuración principal está en **pyproject.toml**.

IDEs y Extensiones

PyCharm	Visual Estudio Code
IDE especializado en Python, con funcionalidades para autocompletado, detección de errores, depuración, gestión de intérpretes, navegación de archivos y funciones, entre otras.	Editor extensible; para trabajar con Python requiere de extensiones como: • Python. Ejecución, depuración. • Pylance. Autocompletado y análisis estático. • Black Formatter. Formateo automático. • Ruff. Análisis y corrección de problemas de estilo. Se debe seleccionar el intérprete para que apunte al ambiente virtual (.venv), y así el editor use las mismas dependencias que el proyecto.

Estructura de un proyecto Python

Una estructura básica puede ser:

```
mi_proyecto/
├── src/
│   ├── mi_proyecto/
│   │   └── __init__.py
└── tests/
```

```

├── pyproject.toml
├── requirements.txt
└── README.md

```

Donde **pyproject.toml** es el archivo central de configuración (parecido a un pom.xml en Java), el cual puede contener metadatos, dependencias y configuración de herramientas; visto más a detalle:

- Nombre y versión del proyecto.
- Dependencias.
- Configuración de herramientas.
- Reglas de formateo.
- Configuración de análisis estático.

El nombre de cada sección va entre corchetes, y contiene pares *clave-valor*.

Por su parte, el archivo **requirements.txt** contiene una lista de dependencias que el proyecto necesita. Aunque para un módulo **Poetry** casi no se utiliza porque centraliza la gestión mediante el pyproject.toml.

¿Por qué no conviene instalar todas las dependencias globalmente?

Porque los proyectos pueden necesitar bibliotecas o versiones diferentes. Los ambientes virtuales aíslan las dependencias de cada proyecto y evitan conflictos. También ayudan a mantener reproducible el entorno.

PEP 8 y PEP 20

Ambas son guías que orientan el diseño en Python, no se sustituyen.

PEP 8	PEP 20: Zen de Python
Define recomendaciones de estilo para escribir código legible y consistente. • Usar nombres descriptivos. • Utilizar cuatro espacios para la indentación. • Separar funciones y clases adecuadamente. • Mantener líneas con una longitud razonable. • Usar snake_case para variables y funciones.	Resume principios de diseño de Python. Puede consultarse ejecutando: <i>python -m this</i> Algunas ideas son: • La legibilidad importa. • Lo explícito es mejor que lo implícito. • Lo simple es mejor que lo complejo. • Debe existir una forma clara de hacer las cosas. • Los errores no deben ignorarse silenciosamente.

Documentación de Excepciones Locales

Una regla de estilo puede tener una excepción justificada por el proyecto. En ese caso, la excepción debe:

- Ser intencional.
- Tener una razón clara.
- Documentarse cerca del código o en la configuración (a manera de comentario).

Herramientas de Calidad y Automatización

- ❖ **Black.** Formateador automático. Modifica el código para aplicar un estilo consistente.
 - Uso en terminal: `"black ."`.
- ❖ **Isort.** Organiza automáticamente los *imports*.
 - Uso: `"isort ."`.
- ❖ **Ruff.** Analiza el código para detectar problemas como *imports* no utilizados, errores comunes, incumplimientos de estilo y problemas que pueden corregirse automáticamente.
 - Uso: `"ruff check ."`.
 - Uso para corrección automática: `"ruff check . --fix"`.
- ❖ **Pre-commit.** Ejecuta comprobaciones automáticas antes de crear un commit. Evita que llegue al repositorio código con errores de formato o estilo, pues puede bloquear el commit hasta que se corrijan. Su configuración suele estar en `.pre-commit-config.yaml`.

NOTAS PARA EL LABORATORIO

Herramientas Black, Isort y Ruff

Dentro del archivo `pyproject.toml` agregué secciones después de `[build-system]` para configurar las herramientas de **black**, **isort** y **ruff**, siendo:

```
[tool.black]
line-length = 88
target-version = ["py312"]
[tool.isort]
profile = "black"
line_length = 88
[tool.ruff]
line-length = 88
target-version = "py312"
[tool.ruff.lint]
select = ["E", "F", "I"]
```

Donde:

- Profile = "black" >> Hace que isort organice los *imports* de manera compatible con Black.
- Select = ["E", "F", "I"] >> Regla que indica a Ruff que revise:
 - E: Estilo relacionado con PEP 8.
 - F: Errores detectables.
 - I: Organización de imports.

Herramienta Pre-commit

Inicialmente creé un archivo `.yaml` para configurar los hooks referentes a Black y Ruff. Dado que *isort* está siendo bloqueado por Windows, ese no fue agregado.

```
repos:
- repo: https://github.com/psf/black-pre-commit-mirror
  rev: 25.1.0
  hooks:
  - id: black
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.11.0
  hooks:
  - id: ruff
    args: [--fix]
  - id: ruff-format
```

Para la herramienta de pre-commit primeramente inicialicé git desde la carpeta del módulo con mi usuario y correo, para después crear un archivo `.gitignore` que excluya el ambiente virtual y archivos temporales (así como `.venv` por ser un ambiente local). Dentro de él:

```
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.ruff_cache/
.cache/
```

Entonces, resumiendo:

- Black formateó el código del archivo "src\ejemplo.py". Entre ellos: espacios, saltos de línea, indentación y presentación general del código.
- Ruff analizó el código y después aplicó correcciones automáticas.
- Isort no pudo ser utilizado debido a políticas de seguridad del equipo.
- Git inicializado de manera local en la carpeta del módulo.
- Pre-commit fue instalado en el repositorio y ejecutó los hooks antes del commit, referentes a black y ruff.
 - Los hooks fueron instalados mediante `poetry run pre-commit install`, lo que modificó el archivo `.pre-commit.config.yaml`.

Flujo desde bash:

1. `poetry run black src`

2. `git add .`
3. `poetry run pre-commit run --all-files`
4. `git add .`
5. `git commit -m "Configura entorno y herramientas del modulo 01"`

Se cumplieron estos objetivos:

- Crear y gestionar un proyecto con Poetry.
- Crear un ambiente virtual aislado.
- Instalar herramientas de calidad.
- Configurar Black, isort y Ruff.
- Aplicar Black y Ruff.
- Crear un repositorio Git local.
- Configurar hooks con pre-commit.
- Corregir infracciones iniciales de estilo.
- Documentar la excepción de isort.

La única parte incompleta es la ejecución de isort, debido a la política de seguridad del equipo corporativo. Como quedó instalada, configurada y documentada, el laboratorio puede considerarse completado con esa excepción registrada.